TEAM 8

Parser

and SDT

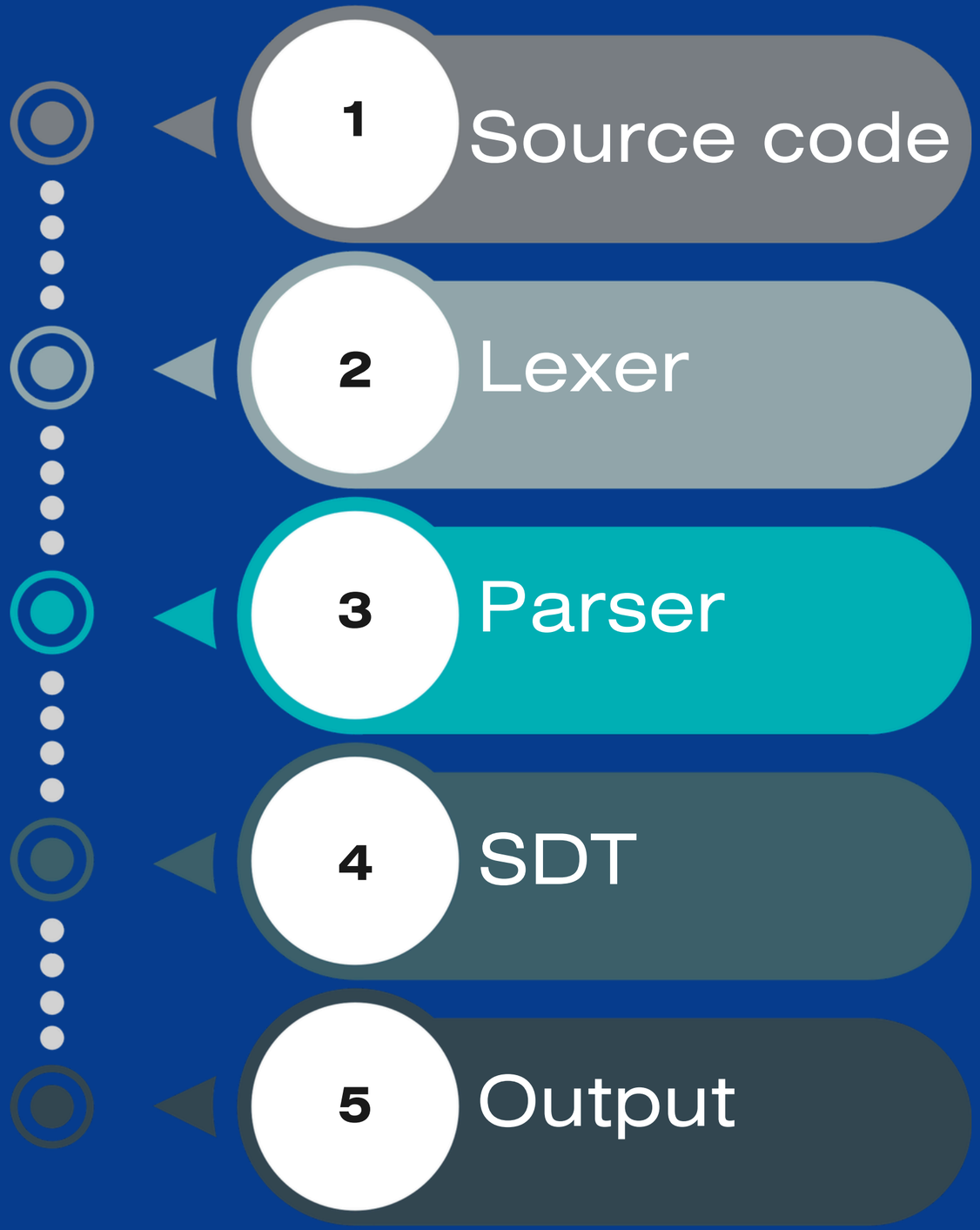
```
int x = 10;
int x = 20;
```

A lexer is not always enough. The parser and SDT are necessary to answer if a program is valid or not without ambiguity

The product

1. Consumes source code of a subset of C
2. Verifies sintax with an LL(1) parser
3. Verifies semantic with an SDT
4. Classifies the output with precision

```
int total = 0;
int main() {
    for (int i = 0; i < 3; i = i + 1) {
        total = total + i * 2;
    }
    if (total == 6) { total = 0; }
    return total;
}
```


1
Source code
2
Lexer
3
Parser
4
SDT
5
Output

Grammar

46 Non-terminals
34 Terminals
95 Productions

Eliminated the left recursion
Factored the common prefixes

First & Follow

```
FIRST(ActualizacionFor) = { OPERATOR_M
FIRST(Asignacion) = { OPERATOR_MINUS O
FIRST(Bloque) = { PUNCTUATION_LBRACE }
```

```
FOLLOW(ActualizacionFor) = { PUNCTUATION_RPAREN }
FOLLOW(Asignacion) = { PUNCTUATION_COMMA PUNCTUATION
FOLLOW(Bloque) = { KEYWORD_CHAR KEYWORD_ELSE KEYWORD
KEYWORD_VOID KEYWORD_WHILE OPERATOR_MINUS OPERATOR_P
```

```
M[Primario, TOKEN_CONSTANT] = 34
M[Primario, TOKEN_IDENTIFIER] = 36
M[Primario, TOKEN_LITERAL] = 35
M[Programa, KEYWORD_CHAR] = 90
M[Programa, KEYWORD_FLOAT] = 90
M[Programa, KEYWORD_INT] = 90
M[Programa, KEYWORD_STRING] = 90
M[Programa, KEYWORD_VOID] = 90
M[Programa, TOKEN_EOF] = 91
M[Sentencia, KEYWORD_CHAR] = 71
M[Sentencia, KEYWORD_FLOAT] = 71
M[Sentencia, KEYWORD_FOR] = 75
M[Sentencia, KEYWORD_IF] = 74
M[Sentencia, KEYWORD_INT] = 71
M[Sentencia, KEYWORD_RETURN] = 73
M[Sentencia, KEYWORD_STRING] = 71
```

0 cells with more than one production

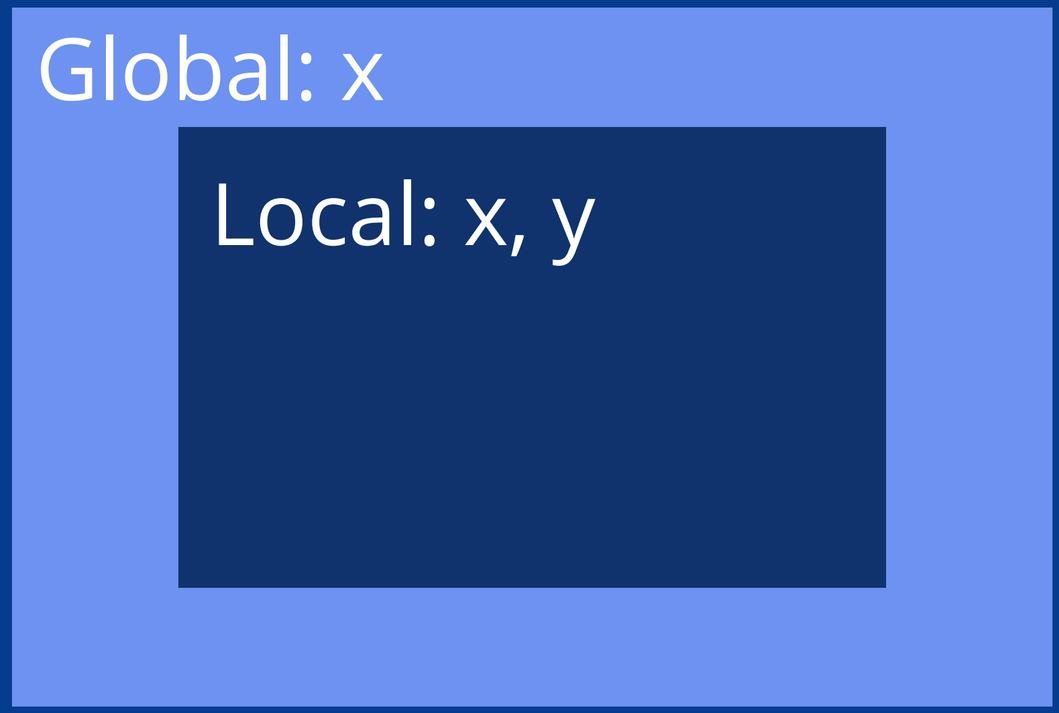
Take a look

int x = 10 ; $

Step	Stack	i/p	Action
.	.	.	.
.	.	.	.
.	.	.	.
10.	$ Programa; ListaVariablesPrima ContinuarExpresion InicioSinId	10;$	R46: InicioSinId → Const
11.	$ Programa; ListaVariablesPrima ContinuarExpresion Const	10;$	match(Const)
12.	$ Programa; ListaVariablesPrima ContinuarExpresion	;$	R45: ContinuarExpresion → TerminoPrima ExpresionPrima
.	.		.
.	.		

SDT

```
int x = 10;
int main() {
    int x = 2;
    int y = 1;
    int y = 3;
}
```



Parsing Success!
SDT error: Error semantico en linea 5: variable 'y' declarada previamente

An example

```
int x = 10;
```

```
========== LEXICAL ANALYSIS ==========
Analisis lexico correcto

========== SYNTAX ANALYSIS ==========
Parsing Success!

========== SEMANTIC ANALYSIS ==========
SDT Verified!
```

```
int x = 10;
int x = 20;
```

```
========== LEXICAL ANALYSIS ==========
Analisis lexico correcto

========== SYNTAX ANALYSIS ==========
Parsing Success!

========== SEMANTIC ANALYSIS ==========
SDT error: Error semantico en linea 2: variable 'x' declarada previamente
```

```
int x = 10;
int y = 21
int z = 2;
```

```
========== LEXICAL ANALYSIS ==========
Analisis lexico correcto

========== SYNTAX ANALYSIS ==========
Error sintactico en linea 3: se encontro 'int' (KEYWORD_INT). Se esperaba: OPERATOR_DIV, O
S, OPERATOR_MULT, OPERATOR_PLUS, PUNCTUATION_COMMA, PUNCTUATION_RPAREN, PUNCTUATION_SEMICO
o: ContinuarExpresion
Parsing error
```

Who is this for?

Our product provides a deterministic solution for the parsing process

- Compiler students: validate exercises with deterministic pass/fail and line-level errors.
- Lightweight C validation: a standalone C++ binary with no external libraries.

The price

Student program

Free

Program License



$15 USD / License

As conclusion

This product is a solid program validator, with the capacity to check a subset of C language and give a precise validation output.

Our team

MURAT CALDERON (PRESENTER)

FRANCISCO REYES (DEVELOPER)

DAN CRUZ (PROJECT MANAGER)

VICTOR BRENA (DESIGNER)

CHRISTOPHER DE LA O (WRITER)